



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ № 2

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура АСОИУ»

Выполнил: студент гр. ИУ5-13Б

(Подпись) (Федяев А. С.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк Ю. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Цель: Разработка консольного приложения на языке C# с применением принципов объектно-ориентированного программирования (ООП) для управления базой данных предметной области «Зоопарки и животные» с использованием СУБД SQLite.

Задачи:

1. Создать классы-модели данных для справочной (Zoo) и основной (Animal) таблиц, реализовав валидацию числовых полей.
2. Сформировать наборы данных в двух файлах csv.
3. Разработать класс DatabaseManager, включающий в себя логику подключения к SQLite и выполнение базовых операций управления данными (добавление, чтение, редактирование, удаление).
4. Спроектировать класс ReportBuilder на основе паттерна Fluent Interface.

Выполнение работы

1. Весь код из различных файлов реализован в namespace HW2_Variant25. Это позволяет не использовать дополнительные using при работе с несколькими файлами.
2. Реализация класса Zoo (файл Animals.cs), описывающего таблицу Zoos. Класс содержит свойства для хранения уникального идентификатора и названия зоопарка (см. рис. 1)

```
0 references
public class Zoo
{
    2 references
    public int Id { get; set; }
    2 references
    public string Name { get; set; }
    2 references
    public Zoo(int id, string name)
    {
        Id = id;
        Name = name;
    }
    0 references
    public Zoo() : this(0, "") { }
    0 references
    public override string ToString() => $"[{Id}] {Name}";
}
```

Рисунок 1 – Код класса Zoo

3. Реализация класса Animals (файл Animals.cs), описывающего основную таблицу «Animals». Содержит свойства для хранения идентификатора, связи с зоопарком (внешний ключ), клички и веса животного (с проверкой, что вес больше или равен 0) (см. рисунок 2).

```
/// <summary>
/// Модель основной таблицы животных
/// </summary>
11 references
public class Animal
{
    3 references
    public int Id { get; set; }
    6 references
    public int ZooId { get; set; }
    7 references
    public string Name { get; set; }

    private double _weight;
    /// <summary>
    /// Масса животного в кг
    /// </summary>
    6 references
    public double Weight
    {
        get => _weight;
        set
        {
            if (value < 0)
                throw new ArgumentException("Масса не может быть отрицательной");
            _weight = value;
        }
    }

    4 references
    public Animal(int id, int zooId, string name, double weight)
    {
        Id = id;
        ZooId = zooId;
        Name = name;
        Weight = weight;
    }

    0 references
    public Animal() : this(0, 0, "", 0) { }
    0 references
    public override string ToString() =>
        $"[{Id}] {Name}, Зоопарк %{ZooId}, Вес: {Weight} кг";
}
```

Рисунок 2 – Код класса Animal

4. Реализация класса DatabaseManager (файл DataBaseManager.cs), содержащего всё взаимодействие с БД. Для этого были реализованы несколько методов класса:
5. Реализация метода InitializeDatabase, создающего таблицы и выполняющего первичный импорт данных из CSV-файлов (см. рис. 3)

```
public class DatabaseManager
{
    private string _connectionString;

    1 reference
    public DatabaseManager(string dbPath)
    {
        _connectionString = $"Data Source={dbPath}";
    }
    /// <summary>
    /// Создает таблицы и выполняет первичный импорт данных из CSV-файлов
    /// </summary>
    /// <param name="zoosCsvPath"></param>
    /// <param name="animalsCsvPath"></param>
    1 reference
    public void InitializeDatabase(string zoosCsvPath, string animalsCsvPath)
    {
        using var conn = new SqlConnection(_connectionString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
            CREATE TABLE IF NOT EXISTS zoos (
                zoo_id INTEGER PRIMARY KEY AUTOINCREMENT,
                zoo_name TEXT NOT NULL
            );
            CREATE TABLE IF NOT EXISTS animals (
                animal_id INTEGER PRIMARY KEY AUTOINCREMENT,
                zoo_id INTEGER NOT NULL,
                animal_name TEXT NOT NULL,
                weight_kg REAL NOT NULL,
                FOREIGN KEY (zoo_id) REFERENCES zoos(zoo_id)
            );";
        cmd.ExecuteNonQuery();

        if (GetAllZoos().Count == 0 && File.Exists(zoosCsvPath))
        {
            ImportZoosFromCsv(zoosCsvPath);
            Console.WriteLine($"[OK] Загружены зоопарки из {Path.GetFileName(zoosCsvPath)}");
        }

        if (GetAllAnimals().Count == 0 && File.Exists(animalsCsvPath))
        {
            ImportAnimalsFromCsv(animalsCsvPath);
            Console.WriteLine($"[OK] Загружены животные из {Path.GetFileName(animalsCsvPath)}");
        }
    }
}
```

Рисунок 3 – Код метода InitializeDatabase(...)

6. Реализация методов ImportZoosFromCsv и ImportAnimalsFromCsv, загружающих записи о зоопарках и животных из CSV-файлов в Базу данных (см. рисунок 4)

```

/// <summary>
/// Загружает записи о зоопарках из CSV-файла в базу данных
/// </summary>
/// <param name="path"></param>
1 reference
private void ImportZoosFromCsv(string path)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    string[] lines = File.ReadAllLines(path);
    for (int i = 1; i < lines.Length; i++)
    {
        string[] parts = lines[i].Split(';');
        if (parts.Length < 2) continue;

        var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO zoos (zoo_id, zoo_name) VALUES (@id, @name)";
        cmd.Parameters.AddWithValue("@id", int.Parse(parts[0]));
        cmd.Parameters.AddWithValue("@name", parts[1]);
        cmd.ExecuteNonQuery();
    }
}

/// <summary>
/// Загружает записи о животных из CSV-файла в базу данных
/// </summary>
/// <param name="path"></param>
1 reference
private void ImportAnimalsFromCsv(string path)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    string[] lines = File.ReadAllLines(path);
    for (int i = 1; i < lines.Length; i++)
    {
        string[] parts = lines[i].Split(';');
        if (parts.Length < 4) continue;

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"INSERT INTO animals (zoo_id, animal_name, weight_kg)
                            VALUES (@zId, @name, @w)";

        cmd.Parameters.AddWithValue("@zId", int.Parse(parts[0]));
        cmd.Parameters.AddWithValue("@name", parts[2]);

        string weightStr = parts[3].Replace(',', '.');
        cmd.Parameters.AddWithValue("@w", double.Parse(weightStr, CultureInfo.InvariantCulture));

        cmd.ExecuteNonQuery();
    }
}

```

Рисунок 4 – Код методов добавления информации в БД при первом запуске

7. Реализация методов GetAllZoos и GetAllAnimals, возвращающих полный список животных и зоопарков из базы данных (см. рисунок 5)

```

/// <summary>
/// Возвращает полный список зоопарков из базы данных
/// </summary>
/// <returns></returns>
3 references
public List<Zoo> GetAllZoos()
{
    var result = new List<Zoo>();
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT zoo_id, zoo_name FROM zoos ORDER BY zoo_id";
    using var reader = cmd.ExecuteReader();
    while (reader.Read())
        result.Add(new Zoo(reader.GetInt32(0), reader.GetString(1)));
    return result;
}
/// <summary>
/// Возвращает полный список животных из базы данных
/// </summary>
/// <returns></returns>
2 references
public List<Animal> GetAllAnimals()
{
    var result = new List<Animal>();
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT animal_id, zoo_id, animal_name, weight_kg FROM animals ORDER BY animal_id";
    using var reader = cmd.ExecuteReader();
    while (reader.Read())
        result.Add(new Animal(reader.GetInt32(0), reader.GetInt32(1), reader.GetString(2), reader.GetDouble(3)));
    return result;
}

```

Рисунок 5 – Код методов GetAllZoos и GetAllAnimals

8. Реализация методов AddAnimal, UpdateAnimal, DeleteAnimal, соответственно добавляющих, изменяющих или удаляющих (по id) животное в базе данных (см. рисунок 6)

```

/// <summary>
/// Добавляет новую запись о животном в базу данных
/// </summary>
/// <param name="animal"></param>
1 reference
public void AddAnimal(Animal animal)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "INSERT INTO animals (zoo_id, animal_name, weight_kg) VALUES (@zId, @name, @w)";
    cmd.Parameters.AddWithValue("@zId", animal.ZooId);
    cmd.Parameters.AddWithValue("@name", animal.Name);
    cmd.Parameters.AddWithValue("@w", animal.Weight);
    cmd.ExecuteNonQuery();
}
/// <summary>
/// Обновляет существующую запись о животном по идентификатору
/// </summary>
/// <param name="animal"></param>
1 reference
public void UpdateAnimal(Animal animal)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "UPDATE animals SET zoo_id=@zId, animal_name=@name, weight_kg=@w WHERE animal_id=@id";
    cmd.Parameters.AddWithValue("@id", animal.Id);
    cmd.Parameters.AddWithValue("@zId", animal.ZooId);
    cmd.Parameters.AddWithValue("@name", animal.Name);
    cmd.Parameters.AddWithValue("@w", animal.Weight);
    cmd.ExecuteNonQuery();
}
/// <summary>
/// Удаляет запись о животном из базы данных по идентификатору
/// </summary>
/// <param name="id"></param>
1 reference
public void DeleteAnimal(int id)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "DELETE FROM animals WHERE animal_id=@id";
    cmd.Parameters.AddWithValue("@id", id);
    cmd.ExecuteNonQuery();
}

```

Рисунок 6 – Код методов для работы с животными в базе данных

9. Реализация метода `GetAnimalById`, который ищет и возвращает запись о животном по идентификатору, и метода `ExecuteQuery`, выполняющего запрос в базу данных и возвращающего названия столбцов и строки данных для отчетов (см. рисунок 7)

```
/// <summary>
/// Ищет и возвращает запись о животном по идентификатору
/// </summary>
/// <param name="id"></param>
/// <returns></returns>
2 references
public Animal GetAnimalById(int id)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT animal_id, zoo_id, animal_name, weight_kg FROM animals WHERE animal_id=@id";
    cmd.Parameters.AddWithValue("@id", id);
    using var reader = cmd.ExecuteReader();
    if (reader.Read())
        return new Animal(reader.GetInt32(0), reader.GetInt32(1), reader.GetString(2), reader.GetDouble(3));
    return null;
}
/// <summary>
/// Выполняет запрос в БД и возвращает названия столбцов и строки данных для отчетов
/// </summary>
/// <param name="sql"></param>
/// <returns></returns>
1 reference
public (string[] columns, List<string[]> rows) ExecuteQuery(string sql)
{
    using var conn = new SqlConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = sql;
    using var reader = cmd.ExecuteReader();

    string[] columns = new string[reader.FieldCount];
    for (int i = 0; i < reader.FieldCount; i++)
        columns[i] = reader.GetName(i);

    var rows = new List<string[]>();
    while (reader.Read())
    {
        string[] row = new string[reader.FieldCount];
        for (int i = 0; i < reader.FieldCount; i++)
            row[i] = reader.GetValue(i)?.ToString() ?? "";
        rows.Add(row);
    }
    return (columns, rows);
}
```

Рисунок 7 – Код методов `GetAnimalById` и `ExecuteQuery`

10. Реализация класса `ReportBuilder` (файл `ReportBuilder.cs`), создающего текстовых отчётов на основе паттерна `Fluent Interface`. Для этого были реализованы промежуточные методы (см. рисунок 8) и терминальные методы `Build` и `Print` (см. рисунок 9).

```

public class ReportBuilder
{
    private DatabaseManager _db;
    private string _sql = "";
    private string _title = "";
    private string[] _headers = Array.Empty<string>();
    private int[] _widths = Array.Empty<int>();
    private bool _numbered = false;
    private string _footer = "";

    1 reference
    public ReportBuilder(DatabaseManager db) { _db = db; }
    /// <summary>
    /// Устанавливает SQL-запрос для получения данных отчета
    /// </summary>
    /// <param name="sql"></param>
    /// <returns></returns>

    3 references
    public ReportBuilder Query(string sql) { _sql = sql; return this; }
    /// <summary>
    /// Задает заголовок формируемого отчета
    /// </summary>
    /// <param name="title"></param>
    /// <returns></returns>

    3 references
    public ReportBuilder Title(string title) { _title = title; return this; }
    /// <summary>
    /// Устанавливает пользовательские названия колонок таблицы
    /// </summary>
    /// <param name="columns"></param>
    /// <returns></returns>

    3 references
    public ReportBuilder Header(params string[] columns) { _headers = columns; return this; }
    /// <summary>
    /// Задает ширину колонок отчета в символах
    /// </summary>
    /// <param name="widths"></param>
    /// <returns></returns>

    3 references
    public ReportBuilder ColumnWidths(params int[] widths) { _widths = widths; return this; }
    /// <summary>
    /// Задаёт сообщение для вывода в конце списка
    /// </summary>
    /// <param name="label"></param>
    /// <returns></returns>

    1 reference
    public ReportBuilder Footer (string label) { _footer = label; return this; }
    /// <summary>
    /// Включает отображение порядковых номеров строк слева
    /// </summary>
    /// <returns></returns>

    1 reference
    public ReportBuilder Numbered() { _numbered = true; return this; }
}

```

Рисунок 8 – Промежуточные методы паттерна *Fluent Interface*


```

/// <summary>
/// Выполняет SQL-запрос и формирует итоговую текстовую строку отчета
/// </summary>
/// <returns></returns>
1 reference
public string Build()
{
    var (columns, rows) = _db.ExecuteQuery(_sql);
    var sb = new StringBuilder();
    if (!string.IsNullOrEmpty(_title))
    {
        sb.AppendLine();
        sb.AppendLine($"=== {_title} ===");
    }
    string[] displayHeaders = _headers.Length > 0 ? _headers : columns;
    int colCount = displayHeaders.Length;
    int numWidth = _numbered ? 5 : 0;

    if (_numbered) sb.Append("%".PadRight(numWidth));
    for (int i = 0; i < colCount; i++)
        sb.Append(displayHeaders[i].PadRight(_widths.Length > i ? _widths[i] : 20));

    sb.AppendLine();

    int totalWidth = numWidth;
    for (int i = 0; i < colCount; i++)
        totalWidth += (_widths.Length > i ? _widths[i] : 20);
    sb.AppendLine(new string('-', totalWidth));

    for (int r = 0; r < rows.Count; r++)
    {
        if (_numbered) sb.Append((r + 1).ToString().PadRight(numWidth));
        for (int c = 0; c < rows[r].Length && c < colCount; c++)
            sb.Append(rows[r][c].PadRight(_widths.Length > c ? _widths[c] : 20));
        sb.AppendLine();
    }
    if (!string.IsNullOrEmpty(_footer))
    {
        sb.AppendLine(new string('-', totalWidth));
        sb.AppendLine($"{{_footer}} {rows.Count}");
    }
    return sb.ToString();
}
/// <summary>
/// Выводит отчёт в консоль
/// </summary>
3 references
public void Print() { Console.Write(Build()); }

```

Рисунок 9 – Терминальные методы паттерна *Fluent Interface*

11. Реализация класса Program (файл Program.cs), реализующего консольный интерфейс и запросы для отчётов. Для этого были реализованы определённые методы.
12. Реализация статического метода Main, содержащего главный цикл консольного меню (см. рисунок 10)

```

static void Main(string[] args)
{
    Console.OutputEncoding = Encoding.UTF8;
    Console.InputEncoding = Encoding.UTF8;

    string zoosCsv = FindFile("zoos.csv");
    string animalsCsv = FindFile("animals.csv");

    if (zoosCsv == null || animalsCsv == null)
    {
        Console.WriteLine("ОШИБКА: Не удалось найти CSV файлы!");
        Console.WriteLine($"Папка запуска: {AppContext.BaseDirectory}");
        Console.WriteLine("Убедись, что файлы называются строго zoos.csv и animals.csv");
        Console.ReadLine();
        return;
    }

    string dbPath = "zoos_data.db";
    var db = new DatabaseManager(dbPath);
    db.InitializeDatabase(zoosCsv, animalsCsv);

    Console.WriteLine();

    string choice;
    do
    {
        Console.WriteLine("--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---");
        Console.WriteLine("1. Показать все зоопарки");
        Console.WriteLine("2. Показать всех животных");
        Console.WriteLine("3. Добавить животное");
        Console.WriteLine("4. Редактировать животное");
        Console.WriteLine("5. Удалить животное");
        Console.WriteLine("6. Отчёты");
        Console.WriteLine("0. Выход");
        Console.Write("Ваш выбор: ");
        choice = Console.ReadLine()?.Trim() ?? "";

        Console.WriteLine();

        switch (choice)
        {
            case "1": ShowZoos(db); break;
            case "2": ShowAnimals(db); break;
            case "3": AddAnimal(db); break;
            case "4": EditAnimal(db); break;
            case "5": DeleteAnimal(db); break;
            case "6": ShowReports(db); break;
            case "0": Console.WriteLine("До свидания!"); break;
            default: Console.WriteLine("Неверный пункт меню."); break;
        }
        Console.WriteLine();
    } while (choice != "0");
}

```

Рисунок 10 – Код метода Main

13. Реализация методов showZoos и showAnimals, выводящих в консоль списки всех зоопарков и всех животных (см. рисунок 11)

```

1 reference
static void ShowZoos(DatabaseManager db)
{
    Console.WriteLine("--- Все зоопарки ---");
    var zoos = db.GetAllZoos();
    foreach (var z in zoos) Console.WriteLine(" " + z);
    Console.WriteLine($"Всего зоопарков: {zoos.Count}");
}
/// <summary>
/// Выводит в консоль список всех зарегистрированных животных
/// </summary>
/// <param name="db"></param>
1 reference
static void ShowAnimals(DatabaseManager db)
{
    Console.WriteLine("--- Все животные ---");
    var animals = db.GetAllAnimals();
    foreach (var a in animals) Console.WriteLine(" " + a);
    Console.WriteLine($"Всего животных: {animals.Count}");
}

```

Рисунок 11 – Код методов showZoos и showAnimals

14. Реализация методов AddAnimal, EditAnimal, DeleteAnimal, соответственно обрабатывающих ввод пользователя с консоли для добавления, изменения или удаления животного в БД (см. рис. 12-14)

```
/// <summary>
/// Обрабатывает ввод пользователя для добавления нового животного
/// </summary>
/// <param name="db"></param>
1 reference
static void AddAnimal(DatabaseManager db)
{
    Console.WriteLine("--- Добавление животного ---");
    Console.WriteLine("Доступные зоопарки:");
    foreach (var z in db.GetAllZoos()) Console.WriteLine("  " + z);

    Console.WriteLine("ID зоопарка: ");
    if (!int.TryParse(Console.ReadLine(), out int zId))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    Console.WriteLine("Имя животного: ");
    string name = Console.ReadLine()?.Trim() ?? "";
    if (name.Length == 0)
    {
        Console.WriteLine("Ошибка: имя не может быть пустым.");
        return;
    }

    Console.WriteLine("Вес (кг): ");
    if (!double.TryParse(Console.ReadLine()?.Replace(',', '.'), NumberStyles.Any, CultureInfo.InvariantCulture, out double weight))
    {
        Console.WriteLine("Ошибка: введите число.");
        return;
    }

    try
    {
        db.AddAnimal(new Animal(0, zId, name, weight));
        Console.WriteLine("Животное успешно добавлено.");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Ошибка: {ex.Message}");
    }
}
```

Рисунок 12 – Код метода AddAnimal

```
/// <summary>
/// Обрабатывает ввод пользователя для редактирования данных животного
/// </summary>
/// <param name="db"></param>
1 reference
static void EditAnimal(DatabaseManager db)
{
    Console.WriteLine("--- Редактирование животного ---");
    Console.WriteLine("Введите ID животного: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var animal = db.GetAnimalById(id);
    if (animal == null)
    {
        Console.WriteLine($"Животное с ID={id} не найдено.");
        return;
    }

    Console.WriteLine($"Текущие данные: {animal}");
    Console.WriteLine("(нажмите Enter, чтобы оставить значение без изменений)");

    Console.WriteLine($"Имя [{animal.Name}]: ");
    string input = Console.ReadLine()?.Trim() ?? "";
    if (input.Length > 0) animal.Name = input;

    Console.WriteLine($"ID зоопарка [{animal.ZooId}]: ");
    input = Console.ReadLine()?.Trim() ?? "";
    if (input.Length > 0 && int.TryParse(input, out int newZId)) animal.ZooId = newZId;

    Console.WriteLine($"Вес [{animal.Weight}]: ");
    input = Console.ReadLine()?.Trim().Replace(',', '.');
    if (input.Length > 0 && double.TryParse(input, NumberStyles.Any, CultureInfo.InvariantCulture, out double newWeight))
    {
        try
        {
            animal.Weight = newWeight;
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine($"Ошибка: {ex.Message}");
            return;
        }
    }

    db.UpdateAnimal(animal);
    Console.WriteLine("Данные обновлены.");
}
```

Рисунок 13 – Код метода EditAnimal

```

/// <summary>
/// Обработывает ввод пользователя для удаления животного
/// (с подтверждением удаления)
/// </summary>
/// <param name="db"></param>
1 reference
static void DeleteAnimal(DatabaseManager db)
{
    Console.WriteLine("---- Удаление животного ----");
    Console.Write("Введите ID животного: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var animal = db.GetAnimalById(id);
    if (animal == null)
    {
        Console.WriteLine($"Животное с ID={id} не найдено.");
        return;
    }

    Console.Write($"Удалить «{animal.Name}»? (да/нет): ");
    string confirm = Console.ReadLine()?.Trim().ToLower() ?? "";
    if (confirm == "да")
    {
        db.DeleteAnimal(id);
        Console.WriteLine("Животное удалено.");
    }
    else
    {
        Console.WriteLine("Удаление отменено.");
    }
}

```

Рисунок 14 – Код метода DeleteAnimal

15. Реализация метода ShowReports, содержащего дополнительное меню для выбора и создания отчётов (см. рисунки 15, 16)

```

/// <summary>
/// Доп. меню для выбора и создания отчётов
/// </summary>
/// <param name="db"></param>
1 reference
static void ShowReports(DatabaseManager db)
{
    string choice;
    do
    {
        Console.WriteLine("--- Отчёты ---");
        Console.WriteLine("1. Животные с названиями зоопарков");
        Console.WriteLine("2. Количество особей в зоопарках");
        Console.WriteLine("3. Средний вес животных по зоопаркам");
        Console.WriteLine("0. Назад");
        Console.Write("Ваш выбор: ");
        choice = Console.ReadLine()?.Trim() ?? "";

        var builder = new ReportBuilder(db);
        switch (choice)
        {
            case "1":
                builder.Query(@"
                    SELECT a.animal_name, z.zoo_name, a.weight_kg
                    FROM animals a
                    JOIN zoos z ON a.zoo_id = z.zoo_id
                    ORDER BY a.animal_name")
                    .Title("Животные по зоопаркам")
                    .Header("Кличка", "Зоопарк", "Вес (кг)")
                    .ColumnWidths(25, 30, 15)
                    .Numbered()
                    .Footer("Всего животных:")
                    .Print();

                break;

```

Рисунок 15 – Первая часть кода метода ShowReports, содержащая меню и первый отчёт

```

        case "2":
            builder.Query(@"
                SELECT z.zoo_name, COUNT(*)
                FROM animals a
                JOIN zoos z ON a.zoo_id = z.zoo_id
                GROUP BY z.zoo_name
                ORDER BY z.zoo_name")
                .Title("Количество особей")
                .Header("Зоопарк", "Кол-во")
                .ColumnWidths(30, 15)
                .Print();
            break;
        case "3":
            builder.Query(@"
                SELECT z.zoo_name, ROUND(AVG(a.weight_kg), 2) AS avg_weight
                FROM animals a
                JOIN zoos z ON a.zoo_id = z.zoo_id
                GROUP BY z.zoo_name
                ORDER BY avg_weight DESC")
                .Title("Средний вес")
                .Header("Зоопарк", "Средний вес (кг)")
                .ColumnWidths(30, 20)
                .Print();
            break;
        case "0":
            break;
        default:
            Console.WriteLine("Неверный пункт.");
            break;
    }
    Console.WriteLine();
} while (choice != "0");
}
}
}

```

Рисунок 16 - Вторая часть кода метода ShowReports, содержащая второй и третий отчёты

16.Примеры работы программы (см. рисунки 17-21)

```

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 1

--- Все зоопарки ---
[1] Московский зоопарк
[2] Вологодский зоопарк
[3] Мир Юрского Периода
[4] Волшебный зоопарк
[5] МГТУ им. Баумана
Всего зоопарков: 5

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 2

--- Все животные ---
[1] Амурский тигр, Зоопарк %1, Вес: 200 кг
[2] Бурый медведь, Зоопарк %1, Вес: 300 кг
[3] Африканский слон, Зоопарк %1, Вес: 4000 кг
[4] Мудрая сова, Зоопарк %1, Вес: 3 кг
[5] Гребнистый крокодил, Зоопарк %1, Вес: 500 кг
[6] Амурский тигр, Зоопарк %2, Вес: 200 кг
[7] Бурый медведь, Зоопарк %2, Вес: 300 кг
[8] Серый волк, Зоопарк %2, Вес: 45 кг
[9] Обыкновенная лисица, Зоопарк %2, Вес: 7 кг
[10] Енот-полоскун, Зоопарк %2, Вес: 8 кг
[11] Тираннозавр Рекс, Зоопарк %3, Вес: 8000 кг
[12] Велоцираптор, Зоопарк %3, Вес: 15 кг
[13] Трицератопс, Зоопарк %3, Вес: 6000 кг
[14] Брахиозавр, Зоопарк %3, Вес: 35000 кг
[15] Гребнистый крокодил, Зоопарк %3, Вес: 500 кг
[16] Белый единорог, Зоопарк %4, Вес: 450 кг
[17] Огнедышащий дракон, Зоопарк %4, Вес: 5000 кг
[18] Горный грифон, Зоопарк %4, Вес: 350 кг
[19] Птица Феникс, Зоопарк %4, Вес: 10 кг
[20] Мудрая сова, Зоопарк %4, Вес: 3 кг
[21] Junior Developer, Зоопарк %5, Вес: 75 кг
[23] Нерабочий код, Зоопарк %5, Вес: 0,01 кг
[24] Енот-полоскун, Зоопарк %5, Вес: 8 кг
[25] Мудрая сова, Зоопарк %5, Вес: 3 кг
[26] Гарри Поттер, Зоопарк %5, Вес: 70 кг
Всего животных: 25

```

Рисунок 17 – Пример работы первых двух функций

```

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 3

--- Добавление животного ---
Доступные зоопарки:
[1] Московский зоопарк
[2] Вологодский зоопарк
[3] Мир Юрского Периода
[4] Волшебный зоопарк
[5] МГТУ им. Баумана
ID зоопарка: 1
Имя животного: Мурка
Вес (кг): 24
Животное успешно добавлено.

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 4

--- Редактирование животного ---
Введите ID животного: 27
Текущие данные: [27] Мурка, Зоопарк %1, Вес: 24 кг
(нажмите Enter, чтобы оставить значение без изменений)
Имя [Мурка]: Клео
ID зоопарка [1]: 4
Вес [24]: 0.3
Данные обновлены.

```

Рисунок 18 – Пример работы третьей и четвёртой функций

```

--- Редактирование животного ---
Введите ID животного: 27
Текущие данные: [27] Мурка, Зоопарк №1, Вес: 24 кг
(нажмите Enter, чтобы оставить значение без изменений)
Имя [Мурка]: Клео
ID зоопарка [1]: 4
Вес [24]: 0.3
Данные обновлены.

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 5

--- Удаление животного ---
Введите ID животного: 27
Удалить «Клео»? (да/нет): да
Животное удалено.

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 6

--- Отчёты ---
1. Животные с названиями зоопарков
2. Количество особей в зоопарках
3. Средний вес животных по зоопаркам
0. Назад
Ваш выбор: 1

```

Рисунок 19 – Пример работы пятой и меню шестой функции

```

=== Животные по зоопаркам ===
% Кличка Зоопарк Вес (кг)
-----
1 Junior Developer МГТУ им. Баумана 75
2 Амурский тигр Московский зоопарк 200
3 Амурский тигр Вологодский зоопарк 200
4 Африканский слон Московский зоопарк 4000
5 Белый единорог Волшебный зоопарк 450
6 Брахиозавр Мир Юрского Периода 35000
7 Бурый медведь Московский зоопарк 300
8 Бурый медведь Вологодский зоопарк 300
9 Велоцираптор Мир Юрского Периода 15
10 Гарри Поттер МГТУ им. Баумана 70
11 Горный грифон Волшебный зоопарк 350
12 Гребнистый крокодил Московский зоопарк 500
13 Гребнистый крокодил Мир Юрского Периода 500
14 Енот-полоскун Вологодский зоопарк 8
15 Енот-полоскун МГТУ им. Баумана 8
16 Мудрая сова Московский зоопарк 3
17 Мудрая сова Волшебный зоопарк 3
18 Мудрая сова МГТУ им. Баумана 3
19 Нерабочий код МГТУ им. Баумана 0,01
20 Обыкновенная лисица Вологодский зоопарк 7
21 Огнедышащий дракон Волшебный зоопарк 5000
22 Птица Феникс Волшебный зоопарк 10
23 Серый волк Вологодский зоопарк 45
24 Тираннозавр Рекс Мир Юрского Периода 8000
25 Трицератопс Мир Юрского Периода 6000
-----
Всего животных: 25

--- Отчёты ---
1. Животные с названиями зоопарков
2. Количество особей в зоопарках
3. Средний вес животных по зоопаркам
0. Назад
Ваш выбор: 2

=== Количество особей ===
Зоопарк Кол-во
-----
Вологодский зоопарк 5
Волшебный зоопарк 5
МГТУ им. Баумана 5
Мир Юрского Периода 5
Московский зоопарк 5

--- Отчёты ---
1. Животные с названиями зоопарков
2. Количество особей в зоопарках
3. Средний вес животных по зоопаркам
0. Назад
Ваш выбор: 3

=== Средний вес ===
Зоопарк Средний вес (кг)
-----
Мир Юрского Периода 9903
Волшебный зоопарк 1162,6
Московский зоопарк 1000,6
Вологодский зоопарк 112
МГТУ им. Баумана 31,2

```

Рисунок 20 – Пример работы отчётов

```

--- Отчёты ---
1. Животные с названиями зоопарков
2. Количество особей в зоопарках
3. Средний вес животных по зоопаркам
0. Назад
Ваш выбор: 0

--- УПРАВЛЕНИЕ ЗООПАРКАМИ (Вариант 25) ---
1. Показать все зоопарки
2. Показать всех животных
3. Добавить животное
4. Редактировать животное
5. Удалить животное
6. Отчёты
0. Выход
Ваш выбор: 0

До свидания!

```

Рисунок 21 – Пример выхода из программы

17. Метаграфовое ситуационное описание. Схема 1 – Модель данных: классы-модели Zoo и Animal (см. рисунок 22). Схема 2 – схема работы класса ReportBuilder для отчёта 1 (см. рисунок 23).

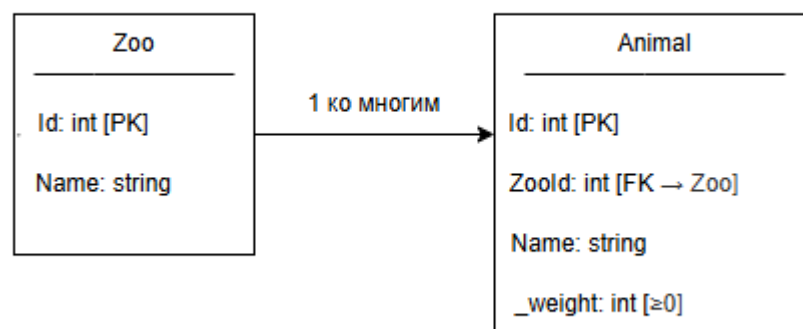


Рисунок 22 – Схема Модели данных, представлена связь «один ко многим»

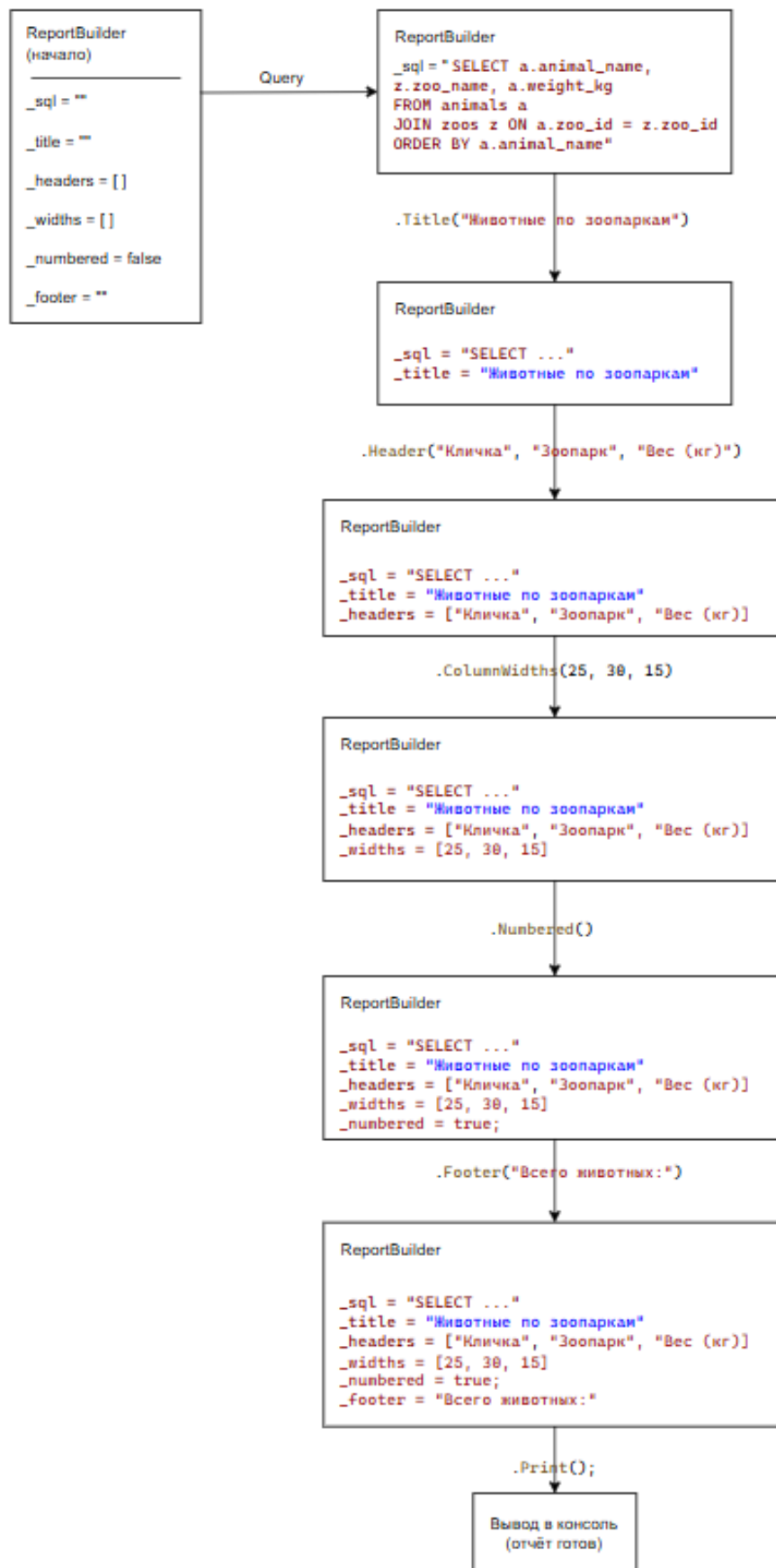


Рисунок 22 – Схема 2 работы класса `ReportBuilder`

18.Дополнительное задание: схема 2 в форме Activity Diagram UML (см. рисунки 23 и 24)

```
@startuml
skinparam monochrome true
title Схема работы класса ReportBuilder

start
:Создание объекта ReportBuilder;
note right
    Состояние:
    _sql = ""
    _title = ""
    _headers = []
    _widths = []
    _numbered = false
    _footer = ""
end note

:Query("SELECT ...");
note right
    Состояние:
    _sql = "SELECT a.animal_name..."
end note

:Title("Животные по зоопаркам");
note right
    Состояние:
    _title = "Животные по зоопаркам"
end note

:Header("Кличка", "Зоопарк", "Вес");
note right
    Состояние:
    _headers = ["Кличка", "Зоопарк", "Вес"]
end note

:ColumnWidths(25, 30, 15);
note right
    Состояние:
    _widths = [25, 30, 15]
end note

:Numbered();
note right
    Состояние:
    _numbered = true
end note

:Footer("Всего записей");
note right
    Состояние:
    _footer = "Всего записей"
end note

:Print();
note right
    Действия терминального метода:
    1. Передача _sql в DatabaseManager
    2. Получение данных (ExecuteQuery)
    3. Отрисовка заголовка и шапки
    4. Форматирование строк данных
    5. Добавление итоговой строки (Footer)
end note

:Вывод готового отчета в консоль;
stop
@enduml
```

Рисунок 23 – Код диаграммы

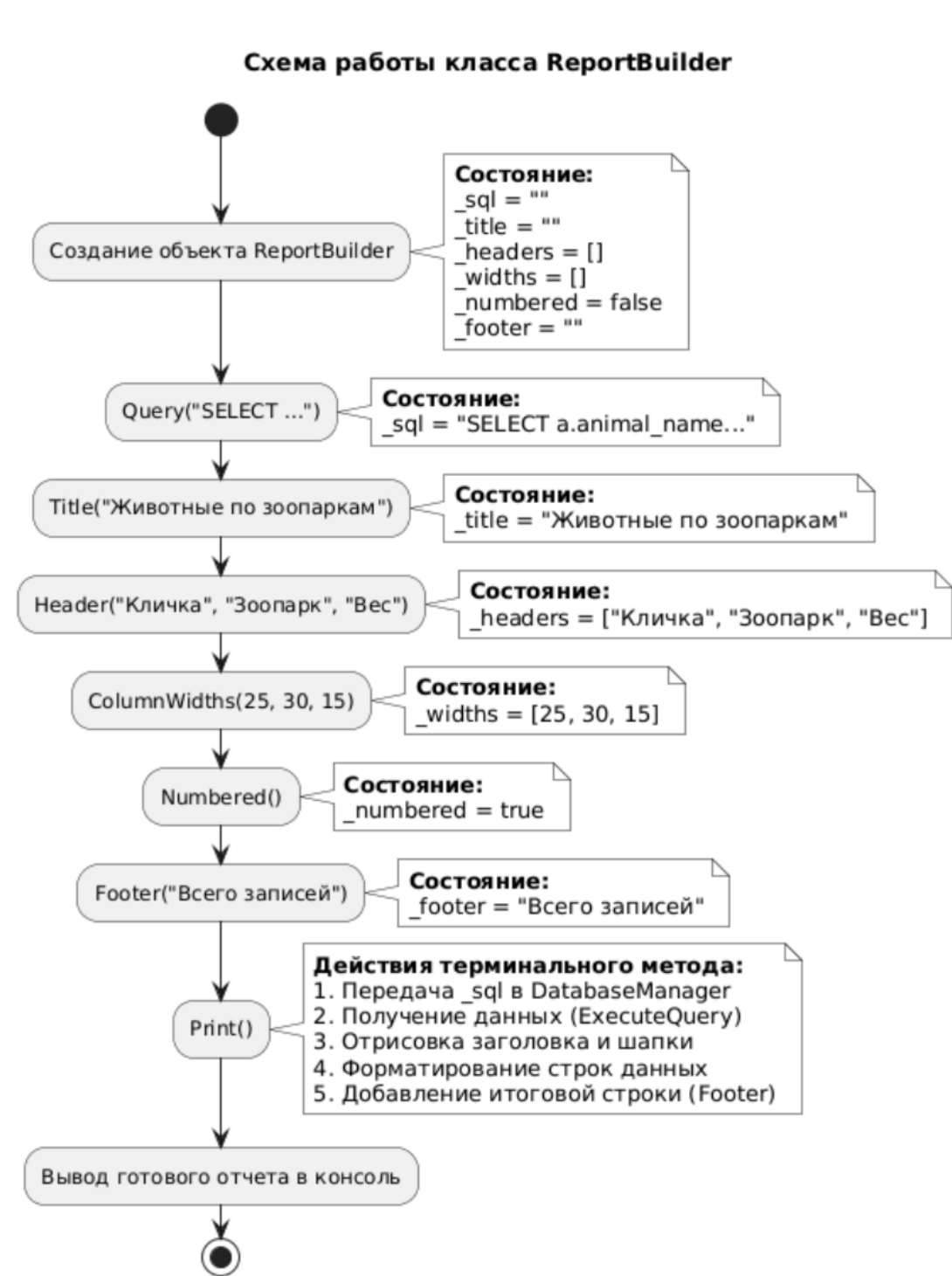


Рисунок 24 – Диаграмма вида Activity Diagram UML для отчёта 1

19. Создание Entity Relationship Diagram для схемы 1 (см. рисунки 25 и 26)

```
@startuml
skinparam monochrome true
hide circle

entity "zoos" {
    * zoo_id : INTEGER <<PK>>
    --
    zoo_name : TEXT
}

entity "animals" {
    * animal_id : INTEGER <<PK>>
    --
    zoo_id : INTEGER <<FK→Zoo>>
    animal_name : TEXT
    weight_kg : REAL
}

zoos ||--o{ animals : "1 ко многим"
@enduml
```

Рисунок 25 – Код Диаграммы

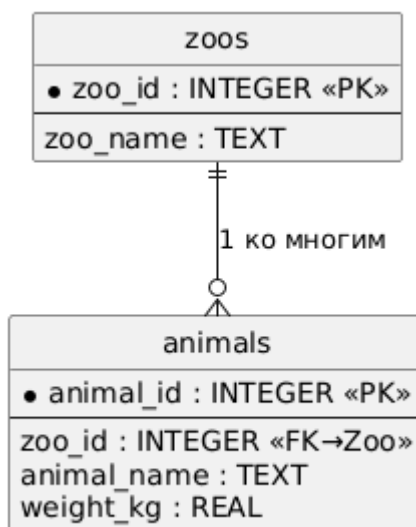


Рисунок 26 – Диаграмма вида Entity Relationship Diagram для схемы 1

20. Реализация диаграммы в форме нотации Чена для схемы 1 (см. рисунки 27 и 28)

```

@startuml
left to right direction
skinparam monochrome true

rectangle "Zoo" as zoo
rectangle "Animal" as animal

usecase "Id [PK]" as z_id
usecase "Name" as z_name

usecase "Id [PK]" as a_id
usecase "ZooId [FK→Zoo]" as a_zid
usecase "Name" as a_name
usecase "Weight [≥ 0]" as a_weight

hexagon "1 ко многим" as rel

zoo -- z_id
zoo -- z_name

animal -- a_id
animal -- a_zid
animal -- a_name
animal -- a_weight

zoo -- rel
rel -- animal
@enduml

```

Рисунок 27 – Код диаграммы

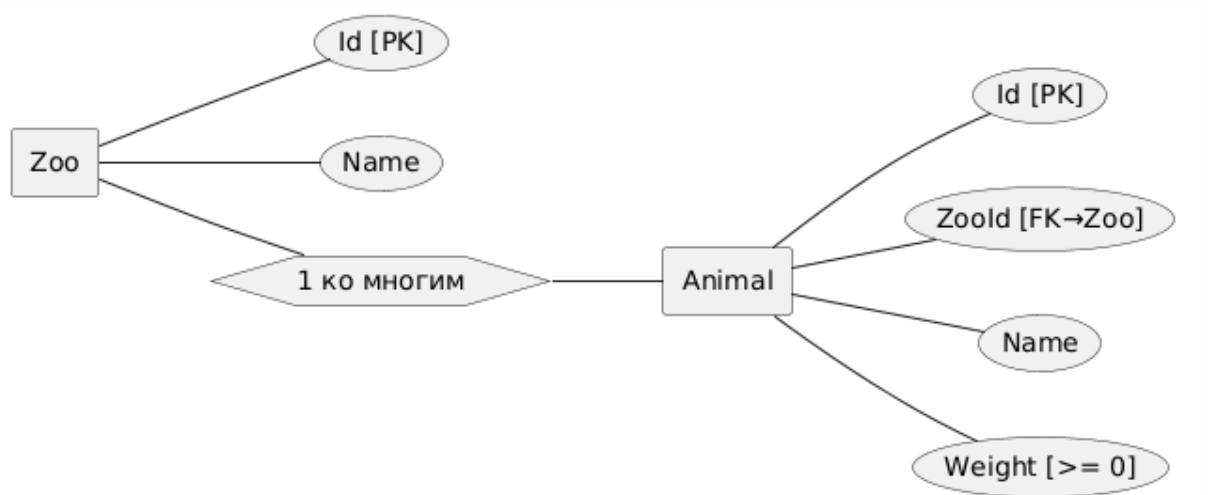


Рисунок 28 – Диаграмма в форме нотации Чена для схемы 1

Вывод: в результате выполнения работы на практике были изучены принципы работы с базами данных, работы с SQLite и языком SQL на примере программы на языке программирования C#.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/103505>
2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/5161>
3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-5-7038-4577-6.
4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002. <http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiya-programmirovaniya.html>
5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.
6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. <http://progbook.ru/c/737-podbelskii-programmiovanie-na-yazyke-si.html>.
7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006. <http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedev-nv-informatika-bazovyy-kurs.html>
8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522. Журнал входит в 1-й уровень Белого списка

Дополнительные материалы

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. <http://razym.ru/naukaobraz/obrazov/181547-aho-a-ulman-d-hopkroft-d-struktury-dannyh-i-algoritmy.html>
2. Дейтел Х.М., Дейтел П.Дж. Как программировать на C++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. <http://rutracker.org/forum/viewtopic.php?t=533181>
4. Джосьютис Н. C++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_c_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в C++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 5-94723-302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. C++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. C/C++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): <https://education.vk.company/students>